

Version Control, Software Testing and Deployment Processes

Detailed Assignment Report

Practical Technology: Git, GitHub, Node.js and Jest

Project: Calculator application with automated unit tests

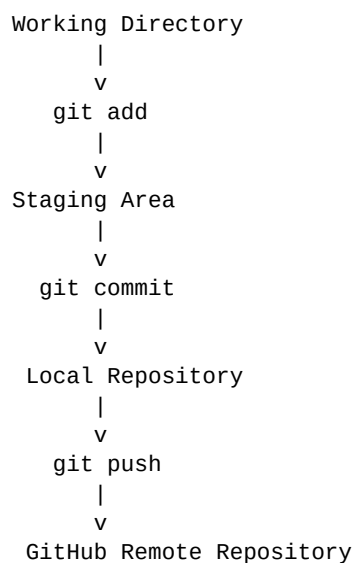
Submitted as part of the software development assignment

1. What is Version Control?

Version control is a system for recording and managing changes to files over time. It allows developers to maintain a history of source code, documents, configuration files and other project resources. Each meaningful change can be recorded as a commit, making it possible to understand what changed, when it changed and which version of the project was available at a particular point in time.

Modern development commonly uses distributed version control systems such as Git. A Git repository stores the project's history locally while services such as GitHub provide remote hosting, collaboration, code review and project management features.

Basic Version Control Workflow



2. Why Use Version Control?

Version control is important because software projects are changed continuously. Without a controlled history, it can be difficult to identify mistakes, restore earlier versions or coordinate work among multiple developers.

- **Change tracking:** Every committed change can be reviewed later.
- **Collaboration:** Multiple developers can work on the same project using branches and merges.
- **Rollback:** A project can be restored to an earlier stable version when a new change causes problems.
- **Backup and remote storage:** A remote Git repository provides another copy of the project's history.
- **Branching:** New features and experiments can be developed separately from stable code.
- **Code review:** Platforms such as GitHub support pull requests and review before changes are merged.

3. Overview of Version Control Systems

Version control systems can be grouped into three broad categories.

Local Version Control

A local system keeps version information on the developer's computer. It provides basic history and rollback but does not naturally support team collaboration across a shared repository.

Centralized Version Control

A centralized system uses a central server that stores the main repository. Developers obtain files from the central server and send changes back to it. This model can simplify centralized administration but depends strongly on the central server.

Distributed Version Control

A distributed system gives each developer a complete repository history. Git is a distributed version control system. Developers can commit locally and synchronize with a remote repository when required.

Git and GitHub

Git is the version control tool, while GitHub is a hosted platform that provides Git repositories plus collaboration features such as pull requests, issues, reviews and automation workflows.

4. Branching Strategies

A branch is an independent line of development. Branching allows developers to make changes without immediately affecting the main stable branch.

- **Main branch:** Contains stable or production-ready code.
- **Feature branches:** Used to develop a particular feature independently.
- **Development branch:** Used in some team workflows to combine work before release.
- **Release branches:** Used to prepare and stabilize a particular release.
- **GitHub Flow:** A lightweight approach in which developers create a short-lived feature branch, make commits, open a pull request, review the changes and merge them into the main branch.

Example Branch Workflow

```
main
|
+---- feature-improvement
      |
      +-- new code
      +-- new tests
      |
      +-- merge
          |
          v
        main
```

5. Merging in Version Control

Merging combines the changes from one branch into another branch. For example, after completing a feature on feature-improvement, a developer can switch to main and merge the feature branch into it.

```
git checkout main
git merge feature-improvement
git push
```

Git normally performs a merge automatically when the changes do not conflict. A merge conflict can occur when two branches modify the same part of a file in incompatible ways. The developer must inspect the conflicting section, choose the correct final content, save the file, stage it and complete the merge.

Why Merging Matters

Merging makes completed work available in the main development line while preserving the history of how the feature was developed. In team environments, pull requests are commonly used so changes can be reviewed before merging.

6. Using Version Control Interfaces

Version control can be used through command-line tools and graphical interfaces.

- **Git command line:** Commands such as `git status`, `git add`, `git commit`, `git branch`, `git merge` and `git push` provide direct control.
- **VS Code Source Control:** The Source Control panel provides a graphical interface for viewing changed files, staging changes, committing and synchronizing.
- **GitHub web interface:** GitHub provides repository browsing, commit history, branches, pull requests, code review and issue management.

Common Git Commands

```
git status
git add .
git commit -m "Describe the change"
git branch
git checkout -b feature-name
git merge feature-name
git push
git pull
```

7. Types of Software Testing

Software testing is the process of evaluating software to find defects and verify that it behaves according to requirements.

- **Unit testing:** Tests a small unit such as a function or method in isolation.
- **Integration testing:** Tests interactions between modules or services.
- **System testing:** Tests the complete application as an integrated system.
- **Acceptance testing:** Checks whether the software meets business or user requirements.
- **Regression testing:** Re-runs tests after changes to ensure existing functionality has not been broken.
- **Performance testing:** Evaluates response time, throughput, resource use and behavior under load.

8. Importance of Automated Testing

Automated testing uses software tools to execute tests and compare actual results with expected results. It is especially useful for repeated checks performed whenever code changes.

- **Early defect detection:** Problems can be identified shortly after a change is introduced.
- **Consistency:** The same test procedure can be repeated reliably.
- **Faster feedback:** A test suite can execute many checks in seconds or minutes.
- **Safer refactoring:** Developers can modify code while using tests to detect regressions.

- **CI/CD support:** Automated tests can run automatically before code is merged or deployed.
- **Reduced manual effort:** Routine verification does not have to be performed manually every time.

9. Implementing Tests in Code

This project uses Node.js with Jest to demonstrate automated unit testing. The application contains four calculator operations and a validation rule for division by zero.

Application Code: app.js

```
function add(a, b) {
  return a + b;
}

function subtract(a, b) {
  return a - b;
}

function multiply(a, b) {
  return a * b;
}

function divide(a, b) {
  if (b === 0) {
    throw new Error("Cannot divide by zero");
  }
  return a / b;
}

module.exports = {
  add,
  subtract,
  multiply,
  divide
};
```

Automated Tests: app.test.js

```
const {
  add,
  subtract,
  multiply,
  divide
} = require("./app");

test("adds two numbers", () => {
  expect(add(5, 3)).toBe(8);
});

test("subtracts two numbers", () => {
  expect(subtract(10, 4)).toBe(6);
});

test("multiplies two numbers", () => {
  expect(multiply(6, 3)).toBe(18);
});

test("divides two numbers", () => {
  expect(divide(20, 4)).toBe(5);
});

test("prevents division by zero", () => {
  expect(() => divide(10, 0)).toThrow(
    "Cannot divide by zero"
  );
});
```

```
    );  
  });
```

Running the Tests

```
npm test
```

A successful run reports that the test suite passed and all five tests passed. This demonstrates that Jest can automatically execute the tests and report failures when actual behavior does not match the expected result.

10. Continuous Deployment Overview

Continuous Deployment (CD) is a software delivery practice in which validated changes can be automatically released to a production environment. It extends continuous integration by connecting automated build and test processes with deployment.

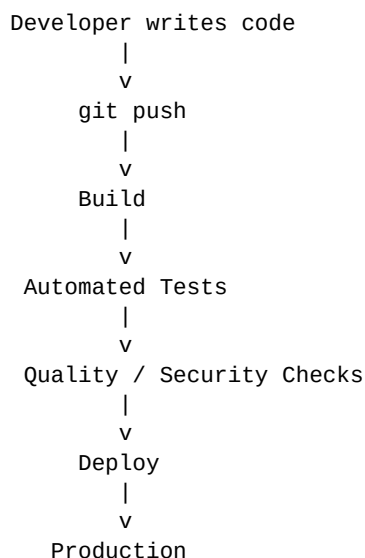
A typical process is: a developer pushes code, an automation system builds the application, automated tests run, and—if the required checks succeed—the application is deployed. Continuous deployment reduces manual release work and supports frequent delivery.

Continuous Integration vs Continuous Deployment

Continuous Integration (CI) focuses on frequently integrating code changes and automatically validating them with builds and tests. **Continuous Deployment** goes further by automatically releasing changes that pass the required checks.

11. Deployment Pipelines Explained

A deployment pipeline is a sequence of automated stages through which software changes pass before reaching users. Each stage provides feedback and can stop an unsafe or defective change from progressing.



A pipeline can include dependency installation, linting, unit tests, integration tests, packaging, security checks, staging deployment and production deployment. The exact stages depend on the project.

12. CI/CD in Practice

GitHub Actions is one practical way to implement CI/CD around a GitHub repository. A workflow can be configured to run when code is pushed or a pull request is opened. The workflow can install dependencies and execute the Jest test suite.

Example GitHub Actions Workflow

```
name: Node.js Tests

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 22

      - run: npm ci
      - run: npm test
```

This workflow is an example of continuous integration: every relevant push or pull request can trigger an automated test run. If a test fails, the team receives feedback before the change is considered ready for release.

Practical Project Workflow

```
Create project
|
Create Git repository
|
Commit source code
|
Create feature branch
|
Add feature + tests
|
Run Jest tests
|
Commit and push branch
|
Merge into main
|
Run CI checks
|
Deploy when approved
```

Practical Demonstration Summary

The GitHub project accompanying this report demonstrates version control through commits, a feature branch and a merge into main. It demonstrates software testing through Jest unit tests for addition, subtraction, multiplication, division and division-by-zero handling. The project can be extended with a GitHub Actions workflow to run these tests automatically whenever changes are pushed or submitted through a pull request.

Expected Repository Structure

```
version-control-testing-deployment/  
■■■ .gitignore  
■■■ README.md  
■■■ app.js  
■■■ app.test.js  
■■■ package.json  
■■■ package-lock.json  
■■■ report/  
    ■■■ Version_Control_Testing_Deployment_Report.pdf
```

Conclusion

Version control, software testing and deployment are closely connected parts of modern software development. Git provides a reliable mechanism for tracking changes, branching and merging work, while GitHub adds collaboration and repository hosting capabilities. Automated testing with frameworks such as Jest provides fast and repeatable feedback about application behavior. CI/CD pipelines connect source control with automated verification and deployment, allowing teams to deliver software more frequently and with greater confidence.

The accompanying calculator project demonstrates these concepts in practice by using Git and GitHub for version control, feature branching and merging, and Jest for automated unit testing.

References

- Git Documentation — <https://git-scm.com/doc>
- GitHub Documentation — <https://docs.github.com/>
- Jest Documentation — <https://jestjs.io/docs/getting-started>
- GitHub Actions Documentation — <https://docs.github.com/en/actions>